
Sistemas Operacionais

Aula 03 - Interação entre tarefas - Parte 3

Rogério Pereira Junior

Instituto Federal de Santa Catarina
campus São José
rogeriopereirajunior@gmail.com

18 de maio de 2026



Problemas Clássicos



- Retratam situações de coordenação típicas em muitos sistemas.
- Descrevem soluções genéricas e eficientes de problemas usuais.
- Exemplos:
 - Produtores/consumidores
 - Leitores/escritores
 - Jantar dos filósofos

Leitura: The Little Book of Semaphores, Allen B. Downey



Produtores/consumidores



- Buffer compartilhado com capacidade para N itens
- Acesso concorrente por dois tipos de tarefas:

Produtor:

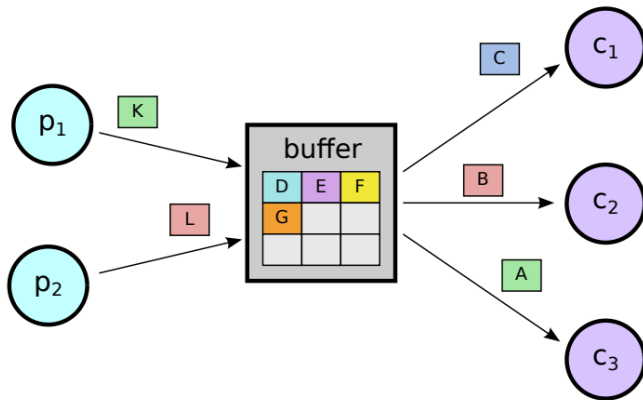
- Produz e deposita itens no buffer
- Se o buffer estiver cheio, espera uma vaga

Consumidor:

- Retira e consome itens do buffer
- Se o buffer estiver vazio, espera um item

Já vimos este cenário para um único PRODUTOR e um único CONSUMIDOR





mailbox $\rightarrow$ Fila de mensagens



Há três aspectos de coordenação a resolver:

- Exclusão mútua no acesso ao buffer
- Bloqueio dos produtores na falta de vagas
- Bloqueio dos consumidores na falta de itens

A solução usa três semáforos:

- **mutex** controla exclusão mútua do buffer (inicia em 1)
- **item** controla itens disponíveis (inicia em 0)
- **vaga** controla vagas disponíveis (inicia em N)



Comportamento básico das tarefas:

Produtor

```
{  
  while (true)  
  {  
    produz_item ;  
  
    coloca_item ;  
  
  }  
}
```

Consumidor

```
{  
  while (true)  
  {  
  
    retira_item ;  
  
    consome_item ;  
  
  }  
}
```



Coordenação do acesso ao buffer: **mutex**

Produtor	Consumidor
<pre>{ while (true) { produz_item ; sem_down (mutex) ; coloca_item ; sem_up (mutex) ; } }</pre>	<pre>{ while (true) { sem_down (mutex) ; retira_item ; sem_up (mutex) ; consome_item ; } }</pre>

- Mutex simples para acessar um por vez a seção crítica, isto é, o buffer.
- Tanto para o consumidor como para o produtor



Coordenação de itens: **item**

Produtor	Consumidor
<pre>{ while (true) { produz_item ; sem_down (mutex) ; coloca_item ; sem_up (mutex) ; sem_up (item) ; } }</pre>	<pre>{ while (true) { sem_down (item) ; sem_down (mutex) ; retira_item ; sem_up (mutex) ; consome_item ; } }</pre>

- Semáforo para deixar as tarefas consumidores dormindo caso não tenha nenhuma tarefa a ser consumida
- Assim que um produtor coloca um item, ele libera (acorda) as tarefas consumidoras que estavam dormindo



Coordenação de vagas: *vaga*

Produtor

```
{  
  while (true)  
  {  
    produz_item ;  
    sem_down (vaga) ;  
    sem_down (mutex) ;  
    coloca_item ;  
    sem_up (mutex) ;  
    sem_up (item) ;  
  }  
}
```

Consumidor

```
{  
  while (true)  
  {  
    sem_down (item) ;  
    sem_down (mutex) ;  
    retira_item ;  
    sem_up (mutex) ;  
    sem_up (vaga) ;  
    consome_item ;  
  }  
}
```

- Semáforo para que o produtor ver se tem espaço disponível no buffer, caso não tenha, suspende as tarefas
- O Consumidor quando retira um item, ele libera uma vaga, então ele sinaliza



```
gcc -Wall prodcons-m2.c -o m3 -lpthread
```



Solução usando variáveis de condição

```
1  mutex mbuf ;           // controla o acesso ao buffer
2  condition item ;       // condição: existe item no buffer
3  condition vaga ;       // condição: existe vaga no buffer
```

```
1  task produtor ()
2  {
3      while (1)
4      {
5          ...             // produz um item
6          lock (mbuf) ;    // obtém o mutex do buffer
7          while (num_itens == N) // enquanto o buffer estiver cheio
8              wait (vaga, mbuf) ; // espera uma vaga, liberando o buffer
9          ...             // deposita o item no buffer
10         signal (item) ;  // sinaliza um novo item
11         unlock (mbuf) ;  // libera o buffer
12     }
13 }
```



```
1 task consumidor ()
2 {
3     while (1)
4     {
5         lock (mbuf) ;           // obtem o mutex do buffer
6         while (num_items == 0) // enquanto o buffer estiver vazio
7             wait (item, mbuf) ; // espera um item, liberando o buffer
8         ...                     // retira o item no buffer
9         signal (vaga) ;         // sinaliza uma vaga livre
10        unlock (mbuf) ;         // libera o buffer
11        ...                     // consome o item retirado do buffer
12    }
13 }
```



Leitores/escritores



Definição

Um conjunto de tarefas acessam uma área de memória compartilhada na qual fazem leituras ou escritas

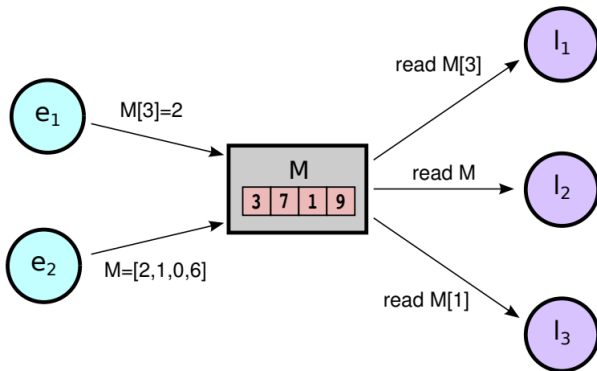
- As leituras podem ser feitas simultaneamente
 - A leitura não altera os dados
 - N leitores podem acessar juntos
- As escritas têm de ser feitas com exclusão mútua
 - A escrita altera os dados
 - Um só escritor pode acessar por vez
 - Leitores não podem acessar durante a escrita



- **Leitores:** são os clientes que entram no aplicativo e consultam o cardápio. Vários podem ler ao mesmo tempo sem atrapalhar uns aos outros, porque apenas estão visualizando.
- **Escritores:** funcionários que precisam atualizar o cardápio — mudar preços, adicionar pratos, retirar itens. Nesse momento, é essencial que ninguém esteja lendo, para não ver informações inconsistentes (por exemplo, um prato que já foi removido).



- Tarefas acessam de forma concorrente uma área de memória compartilhada, na qual podem fazer leituras ou escritas de valores



- Escritas têm de ser feitas com acesso exclusivo à área compartilhada, para evitar condições de disputa.



Comportamento básico das tarefas:

Leitor

```
{  
  while (true)  
  {  
    lê_dados ;  
  
    trata_dados_lidos ;  
  }  
}
```

Esritor

```
{  
  while (true)  
  {  
    obtém_dados ;  
  
    escreve_dados ;  
  }  
}
```



Coordenação do acesso ao buffer: **mutex**

Leitor

```
{  
  while (true)  
  {  
    sem_down (mutex) ;  
    lê_dados ;  
    sem_up (mutex) ;  
    trata_dados_lidos ;  
  }  
}
```

Escritor

```
{  
  while (true)  
  {  
    obtém_dados ;  
    sem_down (mutex) ;  
    escreve_dados ;  
    sem_up (mutex) ;  
  }  
}
```

- Mutex para ler os dados
- Após ler, libera o acesso (area compartilhada) para outro leitor
- O mesmo caso para o escritor

Garante que não vamos ter dois escritores na area compartilhada ao mesmo tempo

Problema: os leitores não podem trabalhar juntos!



- Esta é a Solução Simplista, onde usamos apenas um mutex para tudo.
- É o cenário de exclusão mútua total: se alguém está lendo, ninguém escreve; se alguém está escrevendo, ninguém lê
- É ineficiente: se um leitor está lendo, outro leitor não pode entrar.
- Mesmo se você tiver apenas leitores, eles nunca lerão ao mesmo tempo.
- O sistema fica subutilizado.

Na sua solução simplista com 100 leitores, eles entrarão em fila um por um.
Isso elimina a principal vantagem do paralelismo.



Solução com priorização dos leitores

- Colocamos um contador de leitores iniciando em 0
- Se o leitor é o primeiro, ele pede acesso exclusivo
- Se ele não for o primeiro, ele não pede acesso

Leitor ao entrar:

```
leitores++ ;  
if (leitores == 1)  
    sem_down (mutex) ;
```

Leitor ao sair:

```
leitores-- ;  
if (leitores == 0)  
    sem_up (mutex) ;
```

- Pensem uma sala de aula onde um primeiro aula acendeu a luz
- O próximos alunos não precisam acender
- A mesma ideia na saída!
 - Apenas o ultimo apaga

O código do escritor não muda.



- Precisa de um mutex para proteger o contador: `m_cont`
- Assim que eu obtenho o mutex do contador, eu acesso e modifico o contador
- Depois de fazer a leitura eu liberto o mutex

Leitor ao entrar:

```
sem_down (m_cont) ;  
leitores++ ;  
if (leitores == 1)  
    sem_down (mutex) ;  
sem_up (m_cont) ;
```

Leitor ao sair:

```
sem_down (m_cont) ;  
leitores-- ;  
if (leitores == 0)  
    sem_up (mutex) ;  
sem_up (m_cont) ;
```

- O mesmo quando sair, atualizo o contador e saio.

O código do escritor não muda.



Leitor

```
{  
  while (true)  
  {  
    sem_down (m_cont) ;  
    leitores++ ;  
    if (leitores == 1)  
      sem_down (mutex) ;  
    sem_up (m_cont) ;  
  
    lê_dados ;  
  
    sem_down (m_cont) ;  
    leitores-- ;  
    if (leitores == 0)  
      sem_up (mutex) ;  
    sem_up (m_cont) ;  
  
    trata_dados_lidos ;  
  }  
}
```

mutex: *mutex* do buffer

leitores: contador de
leitores ativos

m_cont: *mutex* do contador

Esta solução prioriza os
leitores. **Por que?**



- O primeiro leitor que chega bloqueia o escritor;
- o último leitor que sai desbloqueia o escritor.

Mais especificamente

- Leitura Simultânea: Se o **rw_mutex** já foi capturado por um leitor, outros 50 leitores podem entrar sem esperar, pois o `if(leitores == 1)` só acontece para o primeiro.
- Segurança do Escritor: O escritor só consegue dar o `sem_wait(&rw_mutex)` quando o último leitor baixar o contador para zero e der o `sem_post`.

Risco: Inanição (Starvation)!



- Leitores podem acessar o recurso simultaneamente sem conflitos, já que apenas consultam.
- Escritores precisam de exclusividade, pois qualquer modificação durante uma leitura ou outra escrita pode gerar inconsistência.

O desafio é sincronizar o acesso para garantir integridade dos dados e, ao mesmo tempo, eficiência.



- **Banco de dados:** várias consultas podem rodar ao mesmo tempo, mas uma atualização precisa bloquear todas para evitar inconsistência.
- **Sistemas de arquivos:** múltiplos usuários podem abrir um documento para leitura, mas apenas um pode editá-lo de cada vez.
- **Aplicações web:** milhares de usuários podem ler posts ou produtos, mas quando alguém edita ou adiciona conteúdo, precisa de exclusividade.



O Jantar dos Filósofos

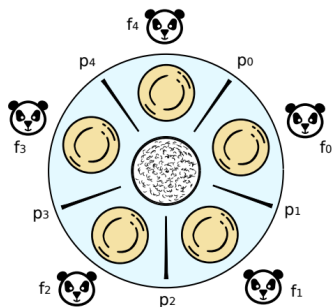




- Cinco filósofos com uma vida bem simples...
 - alternam entre meditar e comer
 - usam uma mesa com 5 lugares
 - cada um tem seu prato
 - os 5 palitos são compartilhados
- Envolve várias tarefas e vários recursos.
- Não há um coordenador central.



- Antes de comer, o filósofo f_i deve pegar os palitos da direita (p_i) e da esquerda (p_{i+1}), um de cada vez
- Após comer, o filósofo devolve os palitos a seus lugares
- Como os palitos são compartilhados, dois filósofos vizinhos não podem comer ao mesmo tempo
- Os filósofos não conversam entre si, nem conhecem os estados uns dos outros
- Não há um coordenador central

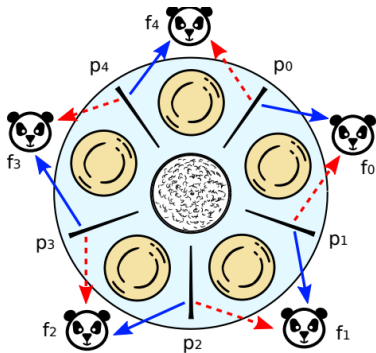


```
1 #define NUMFILO 5
2 semaphore hashi [NUMFILO] ; // palitos são semáforos (iniciam em 1)
3
4 task filosofo (int i)           // filósofo i (entre 0 e 4)
5 {
6     int dir = i ;
7     int esq = (i+1) % NUMFILO ;
8
9     while (1)
10    {
11        meditar () ;
12        down (hashi [dir]) ;      // pega palito direito
13        down (hashi [esq]) ;      // pega palito esquerdo
14        comer () ;
15        up (hashi [dir]) ;         // devolve palito direito
16        up (hashi [esq]) ;        // devolve palito esquerdo
17    }
18 }
```



Problema: risco de impasse!

- Cada filósofo precisa de dois recursos compartilhados (os “hashis”) para comer.
- Se todos os filósofos pegarem ao mesmo tempo apenas um dos hashis e ficarem esperando pelo segundo,
- **NENHUM** deles conseguirá prosseguir — todos ficam bloqueados indefinidamente.



Cada filósofo segura um recurso e espera pelo próximo, formando um ciclo de dependência →
ESPERA CIRCULAR



- **Sistemas operacionais:** processos competindo por CPU, memória ou dispositivos de entrada/saída. Se não houver coordenação, pode ocorrer deadlock.
- **Banco de dados:** transações que bloqueiam tabelas ou linhas e ficam esperando umas pelas outras.
- **Redes de computadores:** pacotes disputando largura de banda ou conexões simultâneas.



- **Planejamento de acesso:** é preciso definir protocolos para evitar que todos tentem usar o mesmo recurso ao mesmo tempo.
- **Prevenção de bloqueios:** sistemas devem prever estratégias para evitar espera circular.
- **Eficiência coletiva:** não basta que cada agente consiga recursos; é necessário que o grupo como um todo funcione sem travar.
- **Equidade:** garantir que nenhum agente fique faminto (starvation), ou seja, sempre esperando sem nunca conseguir usar o recurso.



Atividade



- 1 Pesquisar soluções que priorizem os escritores e soluções que seja justa para os dois lados
- 2 Pesquisa sobre o problema **jantar dos selvagens** e sua solução.
- 3 Implemente uma solução em JAVA para o problema do jantar dos filósofos
 - Faça uma versão usando semáforo e outra usando monitores

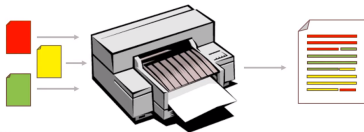


Impasses

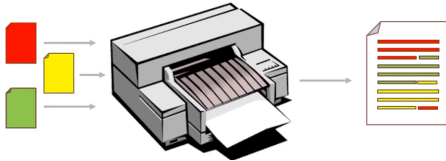


Exemplo Técnico: Impasse de Disco e Impressora

- Imagine dois processos em execução, Processo A e Processo B:
- O Processo A solicita acesso à Impressora para imprimir um relatório e o sistema concede.
- O Processo B solicita acesso ao Disco Rígido para ler dados e o sistema concede.
- O Processo A, ainda segurando a impressora, agora precisa do Disco Rígido para terminar o relatório. Ele fica em estado de espera.
- O Processo B, ainda segurando o disco, agora precisa da Impressora para registrar um log. Ele também entra em espera.
- Resultado: O Processo A não solta a impressora até ter o disco, e o Processo B não solta o disco até ter a impressora. O sistema trava.



- Um deadlock (impasse) acontece quando dois ou mais processos ficam bloqueados para sempre, esperando recursos que nunca serão liberados.



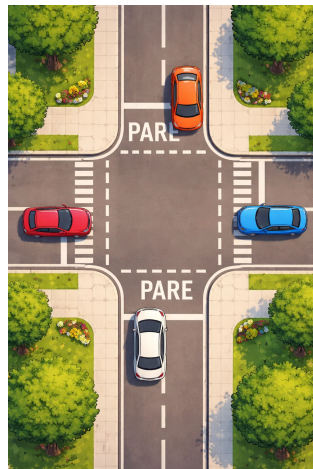
“Cada processo está esperando algo que está nas mãos de outro processo.”

Definição

Um **deadlock** ocorre quando um conjunto de processos fica permanentemente bloqueado: cada processo aguarda por um recurso que já está sendo utilizado por outro processo do mesmo conjunto. Como nenhum deles libera o recurso, todos permanecem em espera indefinida.



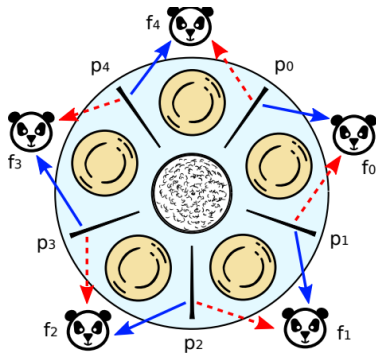
- Imagine um cruzamento de quatro ruas sem semáforo.
 - O carro que vai para o Norte está bloqueado pelo carro que vai para o Leste.
 - O do Leste está bloqueado pelo do Sul.
 - O do Sul pelo do Oeste.
 - E o do Oeste pelo do Norte.
- Nenhum motorista pode avançar sem bater, e nenhum quer dar ré (não preempção).
 - Eles estão em uma espera circular



Se ninguém desistir da sua posição, eles ficarão ali para sempre.



- Cada filósofo precisa de dois recursos compartilhados (os “hashis”) para comer.
- Se todos os filósofos pegarem ao mesmo tempo apenas um dos hashis e ficarem esperando pelo segundo,
- **NENHUM** deles conseguirá prosseguir — todos ficam bloqueados indefinidamente.



Cada filósofo segura um recurso e espera pelo próximo, formando um ciclo de dependência →
ESPERA CIRCULAR



- Dois processos e dois mutexes

- Processo A

- trava recurso R1
- tenta travar R2

- Processo B

- trava recurso R2
- tenta travar R1

Resultado:

- A espera R2
- B espera R1

- Ninguém continua!

```
// Dois recursos compartilhados
pthread_mutex_t R1;
pthread_mutex_t R2;

void* processoA(void* arg) {
    printf("Processo A tentando travar R1...\n");
    pthread_mutex_lock(&R1);
    printf("Processo A travou R1\n");
    sleep(1); // dar chance do processo B executar
    printf("Processo A tentando travar R2...\n");
    pthread_mutex_lock(&R2); // pode ocorrer deadlock
    printf("Processo A travou R2\n");
    pthread_mutex_unlock(&R2);
    pthread_mutex_unlock(&R1);
    return NULL;
}

void* processoB(void* arg) {
    printf("Processo B tentando travar R2...\n");
    pthread_mutex_lock(&R2);
    printf("Processo B travou R2\n");
    sleep(1); // dar chance do processo A executar
    printf("Processo B tentando travar R1...\n");
    pthread_mutex_lock(&R1); // pode ocorrer deadlock
    printf("Processo B travou R1\n");
    pthread_mutex_unlock(&R1);
    pthread_mutex_unlock(&R2);
    return NULL;
}
```



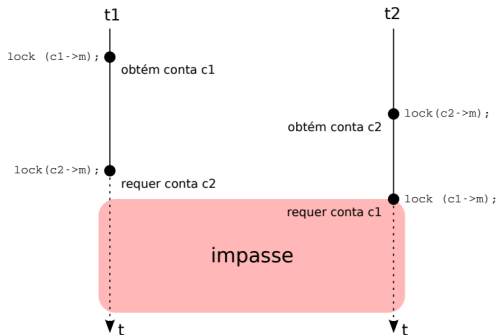
Operação de transferência bancária:

```
1 void transferir (conta_t* contaDeb, conta_t* contaCred, int valor)
2 {
3     sem_down (contaDeb->lock) ;    // obtém acesso a contaDeb
4     sem_down (contaCred->lock) ;    // obtém acesso a contaCred
5
6     if (contaDeb->saldo >= valor)
7     {
8         contaDeb->saldo -= valor ; // debita valor de contaDeb
9         contaCred->saldo += valor ; // credita valor em contaCred
10    }
11    sem_up (contaDeb->lock) ;        // libera acesso a contaDeb
12    sem_up (contaCred->lock) ;        // libera acesso a contaCred
13 }
```



Exemplo de impasse - Conta Bancaria

- A thread t1 pensa: “Só continuo quando c2 for liberada.”
 - Mas: c2 está com t2
- Ao mesmo tempo t2 pensa: “Só continuo quando c1 for liberada.”
 - Mas: c1 está com t1



- Nenhuma thread continua, nenhuma libera os mutexes.
- A situação pode piorar se tivermos mais tarefas querendo acessar essas contas
- Bola de neve, processos vão se aglutinando gerando um impasse maior até travar tudo



Um impasse real (São Paulo SP, 2017)



Para que um impasse ocorra, quatro condições (conhecidas como Condições de Coffman) precisam existir simultaneamente:

- Exclusão Mútua: O recurso só pode ser usado por um processo por vez.
- Posse e Espera: Um processo segura um recurso enquanto espera por outro.
- Não Preempção: O sistema operacional não pode tirar o recurso à força de um processo.
- Espera Circular: O processo A espera pelo B, que espera pelo C, que por sua vez espera pelo A.



Grafos de alocação de recursos

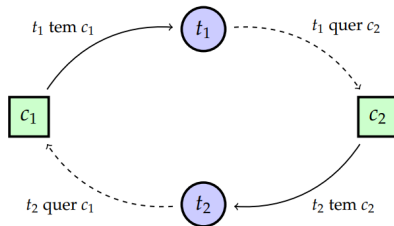
- Permite visualizar a alocação de recursos entre as tarefas de um sistema concorrente
- Permite detectar impasses

○		tarefa
□		recurso
□ → ○		posse de um recurso por uma tarefa (o recurso “pertence” à tarefa)
○ --> □		requisição de um recurso por uma tarefa (a tarefa “quer” o recurso)
□ ● ●		dois recursos do mesmo tipo

- processos → círculos
- recursos → quadrados
- Setas indicam:
 - solicitação;
 - posse de recurso.
- Se existir ciclo → pode haver deadlock.

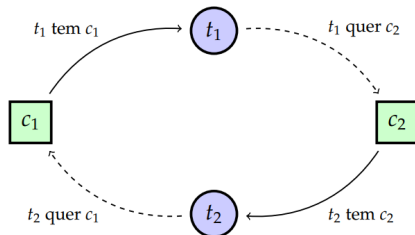


- Percebe-se claramente a dependência cíclica entre tarefas
- Recursos no ciclo $t_1 \rightarrow c_2 \rightarrow t_2 \rightarrow c_1 \rightarrow t_1$, que neste caso evidencia um impasse.



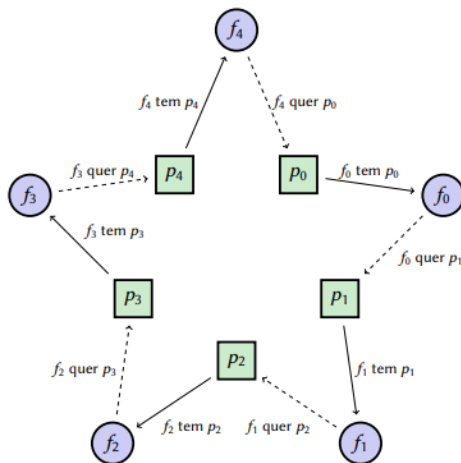
Percebe-se que as quatro condições estão presentes:

- **Exclusão mútua** : as contas são protegidas por semáforos
- **Posse e espera** : tem a conta c_i , requer a conta c_j
- **Não-preempção** : conta só é liberada com `sem_up(c)`
- **Espera circular** : $t_1 \rightarrow c_2 \rightarrow t_2 \rightarrow c_1 \rightarrow t_1$



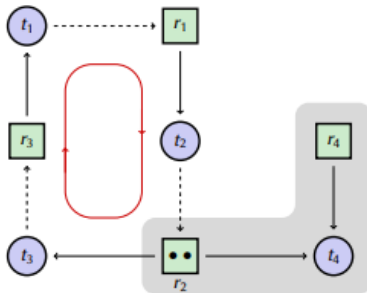
Quatro condições + um recurso de cada tipo $\rightarrow$ impasse!





Múltiplas instâncias de recursos

- Alguns recursos lógicos ou físicos de um sistema computacional podem ter múltiplas instâncias:
- Exemplo, um sistema pode ter duas impressoras idênticas
 - Recurso (impressora) com duas instâncias equivalentes, que podem ser alocadas de forma independente



Sem impasse, mesmo com $t_1 \rightsquigarrow r_1 \rightarrow t_2 \rightsquigarrow r_2 \rightarrow t_3 \rightsquigarrow r_3 \rightarrow t_1$

- t_4 tem r_4 e uma instancia do r_2 . Ela não esta esperando nada!
- Após um tempo ela fará um **up** em r_2 liberando para t_2 .



Estratégias para tratar de impasses:

- **Ignorar** : impasse é um problema do programador!
- **Prevenir** : regras de programação para evitar impasses
- **Impedir** : monitorar o uso de recursos e impedir impasses
- **Detectar** : detectar a ocorrência de impasses e desfazê-los

A mais usada em SOs: ignorar...



A ideia é impedir que uma das 4 condições aconteça.

■ Exclusão mútua:

- Reduzir áreas de exclusão ao mínimo
- Usar técnicas alternativas, como o spooling
- Exemplo: acesso a impressoras em um SO

■ Posse e espera:

- Usar um recurso de cada vez
- Obter todos os recursos antes de iniciar
- Exemplo: na transferência, tratar uma conta por vez
- No jantar dos filósofos eu não consigo: preciso dos dois palitos



■ Não-preempção:

- Poder “arrancar” os recursos dos processos
- Difícil de implementar, pode gerar inconsistências

■ Espera circular:

- Os recursos são ordenados
- As tarefas os solicitam nessa ordem
- Exemplo: acessar as contas segundo os números de conta

Basta quebrar uma das condições!



Exemplo: impedir espera circular

Definir uma ordem global:

- primeiro mutex A
- depois mutex B
- Todos os processos seguem a mesma ordem.
- Assim não há ciclo.



Prevenir impasse com um saleiro!

- Antes de pegar os palitos, o filósofo precisa pegar o saleiro (iniciado com valor 4).
- O saleiro funciona como um “garçom” controlando quantos filósofos podem tentar pegar palitos ao mesmo tempo.

```
1  #define NUMFILO 5
2  semaphore hashi [NUMFILO] ; // palitos são semáforos (iniciam em 1)
3  semaphore saleiro ;         // um semáforo para o saleiro
4
5  task filosofo (int i)       // filósofo i (entre 0 e 4)
6  {
7      int dir = i ;
8      int esq = (i+1) % NUMFILO ;
9
10     while (1)
11     {
12         meditar () ;
13         down (saleiro) ;      // pega saleiro
14         down (hashi [dir]) ;  // pega palito direito
15         down (hashi [esq]) ;  // pega palito esquerdo
16         up (saleiro) ;        // devolve saleiro
17         comer () ;
18         up (hashi [dir]) ;    // devolve palito direito
19         up (hashi [esq]) ;    // devolve palito esquerdo
20     }
21 }
```

- O semáforo permite no máximo 4 filósofos simultaneamente
- Então UM filósofo fica de fora temporariamente.



Estratégia

Acompanhar a alocação dos recursos às tarefas e negar acessos de recursos que possam levar a situações inseguras

Grafo de estados do sistema:

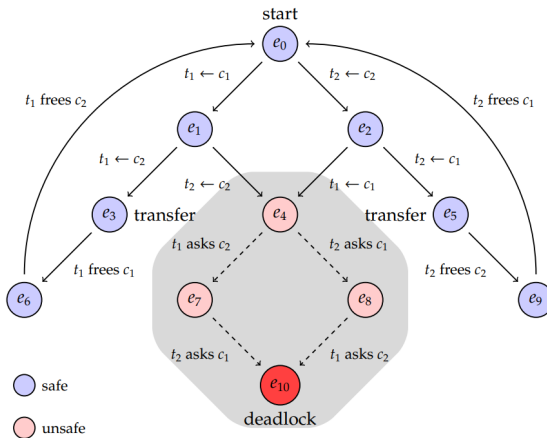
- **Estado:** uma distribuição de recursos entre as tarefas
- **Aresta:** uma alocação ou liberação de recursos

Estados do sistema:

- **Estados seguros:** permitem evoluir aos demais estados
- **Estados inseguros:** somente levam a impasses



Grafo dos estados da conta corrente

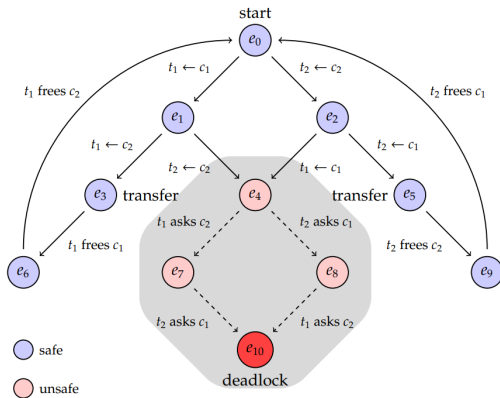


Zona cinza: Conjunto de estados **inseguros**, não consegue-se voltar para os estados **seguros**



Fluxo Feliz

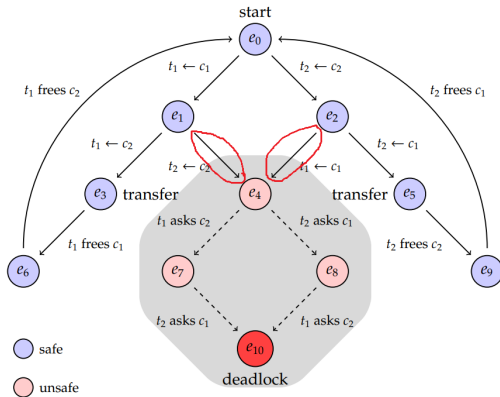
- Estado 0: Nenhum tarefa tem nenhuma conta
- Estado 1: t_1 tem c_1
- Estado 3: t_1 tem c_2 e realiza a transferência
- Estado 6: t_1 libera c_1
- Estado 0: t_1 libera c_2



O problema que precisamos evitar

- Estado 1 para Estado 4
- Estado 2 para Estado 4

Quando a conta 1 tem a tarefa 1, eu não posso deixar a tarefa 2 conseguir a conta 2



- Criado por Edsger Dijkstra.
- O sistema só concede recursos se ainda existir uma sequência segura de execução.
- Analogia bancária
 - Um banco só empresta dinheiro se tiver garantia de conseguir atender todos os clientes futuramente.



Os dois evitam deadlock, mas de maneiras diferentes.

Saleiro

- “Nem todo mundo pode competir ao mesmo tempo.”
- O saleiro simplesmente aplica uma regra fixa: “Só 4 podem entrar.”
- Ele NÃO:
 - analisa estados;
 - calcula segurança;
 - prevê o futuro.

Banqueiro

- “Se eu conceder esse recurso agora, o sistema continuará seguro?”
- “Vou calcular se consigo atender todos antes de liberar.”



O saleiro evita deadlock impedindo uma condição de Coffman:

Espera Circular

Como ele faz isso?

- Existem N filósofos
- Apenas $N - 1$ podem competir

Resultado:

- sempre sobra recurso;
- alguém termina;
- recursos são liberados;
- o sistema continua.



- O algoritmo do banqueiro é mais inteligente.
- Ele NÃO usa regra fixa.
- Antes de conceder um recurso, ele pergunta:
“Se eu entregar esse recurso agora, ainda existe uma sequência segura?”
- O algoritmo verifica:
 - recursos disponíveis;
 - quanto cada processo já possui;
 - quanto ainda pode pedir;
 - se alguém ainda consegue terminar.
- Se existir sequência segura: recurso é concedido.
- Senão: recurso é negado.



- Técnicas de impedimento de impasses exigem conhecimento prévio sobre o comportamento das tarefas.
- É necessário saber:
 - Quais recursos cada tarefa acessará;
 - Quantas instâncias de cada recurso serão necessárias;
 - A ordem de acesso aos recursos.

Limitação na prática

Essas técnicas são pouco utilizadas, pois raramente é possível conhecer todos os recursos e a ordem de acesso com antecedência.



Observar o sistema e, quando ocorrer um impasse, resolvê-lo.

Como **detectar** impasses?

- Manter um grafo de alocação de recursos:
 - Atualizar a cada requisição/alocação/liberação
 - Detectar a formação de ciclos no grafo
 - Pode exigir muito processamento
- Monitorar nível de atividade do sistema:
 - Analisar tarefas suspensas há muito tempo



Como **resolver** impasses?

- **Eliminar tarefas** de modo a romper os ciclos
 - Eliminar a mais nova? A mais antiga? A menos prioritária?
- **Retroceder tarefas**, retornando a um estado seguro
 - Técnica de rollback usada em bancos de dados
 - São necessários checkpoints periódicos
 - Algumas operações não podem ser desfeitas
 - interações com o usuário
 - envio de pacotes de rede



- Estratégia utilizada em muitos sistemas operacionais.
- Considera que o impasse é responsabilidade do programador.
- Conhecida como:
 - **“Algoritmo do Avestruz”**
 - “enfia a cabeça na areia” como o avestruz, fingindo que o problema não existe.
- **Justificativa:** Deadlocks são considerados raros, e os mecanismos de prevenção/detecção podem reduzir desempenho.
- Usado em:
 - Muitos sistemas **UNIX**;
 - **Linux** em diversas situações.

Observação Importante

Essa abordagem simplifica o sistema, mas pode deixar aplicações vulneráveis a travamentos caso o programador não trate corretamente os recursos.



Deadlock vs Starvation

- Deadlock: Processo nunca continua porque há espera circular.
- Starvation: Processo nunca executa porque outros sempre recebem prioridade.

Deadlock vs Race Condition

- Deadlock: Problema de bloqueio.
- Race Condition: Problema de concorrência e inconsistência de dados.

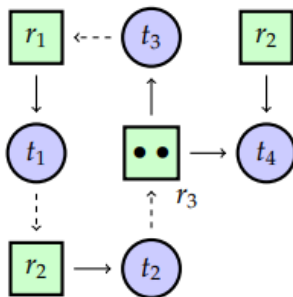
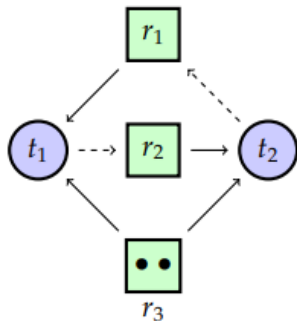


Análise a solução de impasses no problema do jantar dos filósofos onde alteramos a ordem de escolha dos hashis para o último filósofo.



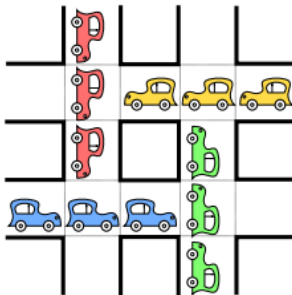
Atividade 2

Nos grafos de alocação de recursos da figura a seguir, indique o(s) ciclo(s) onde existe um impasse:



Atividade 3

- A figura a seguir representa uma situação de impasse em um cruzamento de trânsito.
- Todas as ruas têm largura para um carro e sentido único.
 - 1 Mostre que as quatro condições necessárias para a ocorrência de impasses estão presentes nessa situação.
 - 2 Em seguida, defina uma regra simples a ser seguida por cada carro para evitar essa situação; **Regras envolvendo algum tipo de informação centralizada não devem ser usadas**



- MAZIERO, C. Sistemas Operacionais: Conceitos e Mecanismos. Editora da UFPR, 2019. 456 p. ISBN 978-85-7335-340-2
- SILBERSCHATZ, Abraham; GALVIN, Peter Baer; GAGNE, Greg. Fundamentos de Sistemas: Operacionais. Princípios Básicos. Grupo Gen-LTC, 2000.
- MACHADO, Francis Berenger; MAIA, Luiz Paulo. Arquitetura de sistemas operacionais. LTC, 2004.

